



ok, peux tu écrire le programme Python correspondant:

Voici un exemple complet en Tkinter qui implémente le comportement décrit (sélection de fichier .bloc dans bloc/user ou bloc/system + filtrage par tags).

```
import os
import pickle
import tkinter as tk
from tkinter import ttk
from tkinter import filedialog

# Racines possibles pour les fichiers .bloc
BASE_DIR_USER = os.path.join("bloc", "user")
BASE_DIR_SYSTEM = os.path.join("bloc", "system")

class BlocFileSelector(tk.Toplevel):
    def __init__(self, master, mode_repertoire="user", title="Sélection d'un fichier .t
        super().__init__(master)
        self.title(title)

        # Détermination du répertoire racine
        if mode_repertoire == "system":
            self.base_dir = BASE_DIR_SYSTEM
        else:
            self.base_dir = BASE_DIR_USER

        # Données internes
        self.all_files = []      # liste de dicts: {"path":..., "name":..., "tags":set
        self.all_tags = set()    # ensemble de tous les tags trouvés
        self.tag_vars = {}       # tag -> tk.BooleanVar
        self.selected_file = None # chemin du fichier choisi (ou None)

        # Construction de l'UI
        self._build_ui()

        # Chargement des fichiers et tags
        self._scan_bloc_files()
        self._init_tag_selection()
        self._update_file_list()

        # Comportement modal
        self.transient(master)
        self.grab_set()
        self.protocol("WM_DELETE_WINDOW", self._on_cancel)
```

```

# ----- Construction de l'interface -----

def _build_ui(self):
    # Frame principale
    main_frame = ttk.Frame(self)
    main_frame.pack(fill=tk.BOTH, expand=True, padx=10, pady=10)

    # Frame tags
    tags_frame = ttk.LabelFrame(main_frame, text="Tags disponibles")
    tags_frame.pack(side=tk.LEFT, fill=tk.BOTH, expand=False, padx=(0, 10))

    self.tags_canvas = tk.Canvas(tags_frame, borderwidth=0, highlightthickness=0)
    self.tags_scrollbar = ttk.Scrollbar(tags_frame, orient="vertical", command=self.tags_inner.yview)
    self.tags_inner = ttk.Frame(self.tags_canvas)

    self.tags_inner.bind(
        "<Configure>",
        lambda e: self.tags_canvas.configure(scrollregion=self.tags_canvas.bbox("all"))
    )
    self.tags_canvas.create_window((0, 0), window=self.tags_inner, anchor="nw")
    self.tags_canvas.configure(yscrollcommand=self.tags_scrollbar.set)

    self.tags_canvas.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)
    self.tags_scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

    # Frame fichiers
    files_frame = ttk.LabelFrame(main_frame, text="Fichiers .bloc")
    files_frame.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

    self.files_listbox = tk.Listbox(files_frame, height=15)
    self.files_listbox.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

    files_scrollbar = ttk.Scrollbar(files_frame, orient="vertical", command=self.files_listbox.yview)
    files_scrollbar.pack(side=tk.RIGHT, fill=tk.Y)
    self.files_listbox.configure(yscrollcommand=files_scrollbar.set)

    # Frame boutons
    buttons_frame = ttk.Frame(self)
    buttons_frame.pack(fill=tk.X, padx=10, pady=(0, 10))

    btn_ok = ttk.Button(buttons_frame, text="Ouvrir", command=self._on_ok)
    btn_cancel = ttk.Button(buttons_frame, text="Annuler", command=self._on_cancel)

    btn_ok.pack(side=tk.RIGHT, padx=(5, 0))
    btn_cancel.pack(side=tk.RIGHT)

# ----- Chargement des fichiers et tags -----

def _scan_bloc_files(self):
    """
    Parcourt le répertoire base_dir, charge les fichiers .bloc,
    récupère les tags et remplit self.all_files et self.all_tags.
    """
    if not os.path.isdir(self.base_dir):
        return

```

```

for root, dirs, files in os.walk(self.base_dir):
    for fname in files:
        if not fname.lower().endswith(".bloc"):
            continue
        full_path = os.path.join(root, fname)
        tags = set()
        try:
            with open(full_path, "rb") as f:
                structure = pickle.load(f) # [web:20]
                header = getattr(structure, "header", None)
                if isinstance(header, dict):
                    kw = header.get("key_word", [])
                    if isinstance(kw, (list, tuple, set)):
                        tags = set(str(t) for t in kw)
        except Exception:
            # Fichier illisible : on l'ignore
            continue

        self.all_files.append({
            "path": full_path,
            "name": fname,
            "tags": tags
        })
        self.all_tags.update(tags)

# Création des checkboxes pour les tags (triés)
for tag in sorted(self.all_tags):
    var = tk.BooleanVar(value=False)
    cb = ttk.Checkbutton(self.tags_inner, text=tag, variable=var,
                        command=self._on_tag_change)
    cb.pack(anchor="w")
    self.tag_vars[tag] = var

def _init_tag_selection(self):
    """
    Initialise la sélection des tags :
    - si "main" est présent, il est coché
    - sinon si "basic" est présent, il est coché
    - sinon aucun tag sélectionné
    """
    if "main" in self.tag_vars:
        self.tag_vars["main"].set(True)
    elif "basic" in self.tag_vars:
        self.tag_vars["basic"].set(True)

# ----- Logique de filtrage -----

def _get_selected_tags(self):
    """
    Retourne l'ensemble des tags actuellement sélectionnés.
    """
    selected = {tag for tag, var in self.tag_vars.items() if var.get()}
    return selected

def _update_file_list(self):
    """

```

```

Met à jour la listbox en fonction des tags sélectionnés.
Un fichier est affiché s'il a au moins un des tags sélectionnés.
Si aucun tag n'est sélectionné, ici on choisit d'afficher aucun fichier.
"""

selected_tags = self._get_selected_tags()
self.files_listbox.delete(0, tk.END)

for f in self.all_files:
    if not selected_tags:
        # Choix : aucun fichier si aucun tag sélectionné
        continue
    # Intersection non vide ?
    if f["tags"] & selected_tags:
        self.files_listbox.insert(tk.END, f["path"])

def _on_tag_change(self):
    """
    Callback quand un tag est coché/décoché.
    """
    self._update_file_list()

# ----- Gestion validation / annulation -----

def _on_ok(self):
    """
    Récupère le fichier sélectionné et ferme la fenêtre.
    """
    selection = self.files_listbox.curselection()
    if not selection:
        self.selected_file = None
    else:
        # On stocke le chemin complet (c'est ce qui est dans la listbox)
        self.selected_file = self.files_listbox.get(selection[0])
    self.destroy()

def _on_cancel(self):
    """
    Annulation : aucune sélection.
    """
    self.selected_file = None
    self.destroy()

def select_bloc_file(mode_repertoire="user", title="Sélection d'un fichier .bloc"):
    """
    Fonction de haut niveau équivalente à filedialog.askopenfilename,
    mais avec filtrage par tags sur des fichiers .bloc dans bloc/user ou bloc/system.

    Retourne :
    - chemin complet du fichier sélectionné (str)
    - ou None si l'utilisateur annule.
    """
    root = tk.Tk()
    root.withdraw() # on cache la fenêtre principale

    dialog = BlocFileSelector(root, mode_repertoire=mode_repertoire, title=title)

```

```

root.wait_window(dialog)

root.destroy()
return dialog.selected_file

# Exemple d'utilisation directe
if __name__ == "__main__":
    fichier = select_bloc_file(mode_repertoire="user",
                              title="Choisissez un fichier .bloc (user)")

    if fichier:
        print("Fichier sélectionné :", fichier)
    else:
        print("Aucun fichier sélectionné.")

```

Ce code utilise Tkinter pour créer une fenêtre modale, parcourt `bloc/user` ou `bloc/system`, charge les objets avec `pickle.load` et filtre la liste en fonction des tags cochés, avec présélection éventuelle de `main` ou `basic`.^[1] ^[2]

**

1. <https://www.pythontutorial.net/tkinter/tkinter-open-file-dialog/>
2. <https://realpython.com/ref/stdlib/pickle/>
3. https://www.youtube.com/watch?v=k-QxYTr_k_O
4. <https://stackoverflow.com/questions/61758460/tkinter-filedialog-askopenfilename-function>
5. <https://stackoverflow.com/questions/55010224/how-to-load-object-in-pickle>
6. <https://stackoverflow.com/questions/64525845/how-to-change-a-list-depending-on-tkinter-checkbutton>
7. <https://python.developpez.com/tutoriels/cours-python-uni-paris7/>
8. <https://stackoverflow.com/questions/10993089/opening-and-reading-a-file-with-askopenfilename>
9. <https://realpython.com/videos/using-pickle-module/>
10. <https://covrisolutions.wordpress.com/2019/01/29/tkinter-examples-5-check-buttons/>